

TinyVLM: A Distilled and Quantized Vision-Language Model for Efficient Food Image-Text Retrieval in TinyML Settings

Podakanti Satyajith Chary

Department of Engineering Science
IIT Hyderabad
Hyderabad, India
es25resch11002@iith.ac.in

Pithani Teja Venkata Ramana Kumar

Department of Biomedical Engineering
IIT Hyderabad
Hyderabad, India
pithaniteja@alumni.iith.ac.in

Nagarajan Ganapathy

Department of Biomedical Engineering
IIT Hyderabad
Hyderabad, India
gnagarajan@bme.iith.ac.in

Abstract—The convergence of Artificial Intelligence and the Internet of Things (AIoT) enables intelligent applications on resource-constrained edge devices. However, the prohibitive size and computational cost of state-of-the-art Vision-Language Models (VLMs) present a significant barrier to on-device deployment. This paper presents TinyVLM, a compact and efficient VLM tailored for fine-grained food recognition in TinyML settings. Built on the modular nanoVLM framework, our approach applies a two-stage compression strategy: knowledge distillation to produce a lightweight student model followed by dynamic post-training quantization. On the Dishcovery VLM MetaFood Challenge validation set, the teacher model achieves 84.6% retrieval accuracy whereas TinyVLM (distilled + quantized) attains 76.8% which is a modest loss in accuracy for a large reduction in footprint. In preliminary on-device benchmarking on an NVIDIA Jetson platform, the quantized TinyVLM achieved a model size of 111.1 MB and an average inference latency of 1.39 s, compared to the teacher ($\approx$ 847 MB, 1.97 s). These preliminary on-device results demonstrate the practical feasibility of deploying TinyVLM in edge scenarios.

Index Terms—Vision-Language Model, TinyML, AIoT, Knowledge Distillation, Dynamic Quantization, Model Compression, Food Recognition

I. INTRODUCTION

The proliferation of Internet of Things (IoT) devices has spurred demand for intelligent, on-device processing to enable real-time applications in domains like healthcare, smart homes, and agriculture. In the food sector, Vision-Language Models (VLMs) offer promising capabilities for tasks such as image-text matching, which can power personalized nutrition tracking in wearables or automated recipe suggestions in smart refrigerators. However, state-of-the-art VLMs, such as those based on large vision transformers (ViTs) and language models (LMs), require substantial computational resources, rendering them unsuitable for TinyML deployments on edge devices with limited memory and power budgets [1], [2].

This paper investigates the adaptation of a powerful VLM for a specialized, resource-constrained task: fine-grained food image-text retrieval. Our work is motivated by the following research questions:

- **RQ1:** How effectively can a systematic pipeline of knowledge distillation and post-training quantization compress a modern VLM for a specialized domain like food recognition, without catastrophic loss of performance?
- **RQ2:** What is the quantitative trade-off between model size, inference speed, and fine-grained retrieval accuracy when applying this compression pipeline?

To address these challenges, we introduce TinyVLM, a lightweight VLM adapted for fine-grained food image-text retrieval. We present a systematic pipeline, built upon the modular ‘nanoVLM’ framework [3], that effectively compresses a powerful VLM for on-device use. Our primary contribution is the holistic application and rigorous evaluation of this multi-stage compression strategy - combining architectural down-scaling, knowledge distillation, and dynamic quantization—to create a model that balances performance and efficiency for a real-world AIoT task. Our pipeline, illustrated in Fig. 1, drastically reduces the model’s footprint from 888 MB to 338 MB, making it suitable for on-device deployment.

II. RELATED WORK

A. Large-Scale Vision-Language Models

Recent years have seen the emergence of powerful VLMs that learn joint embeddings from web-scale data. Models like CLIP [4] and ALIGN [5] demonstrated that a simple contrastive objective can lead to impressive zero-shot generalization. Our work builds upon this paradigm by using pre-trained backbones designed for this purpose. Specifically, we use SigLIP [6], a vision model trained with a sigmoid loss that has shown strong performance, and SmoLLM [7], a compact yet capable language model. We integrate these within the ‘nanoVLM’ educational framework [3], which provides a modular structure for VLM construction.

B. Model Compression for Edge AI

The need to run AI on edge devices has spurred research into model compression. Major strategies include network pruning, low-rank factorization, compact network design,

TinyVLM Architecture: Efficient Food Retrieval

A Three-Stage Teacher-Student Pipeline

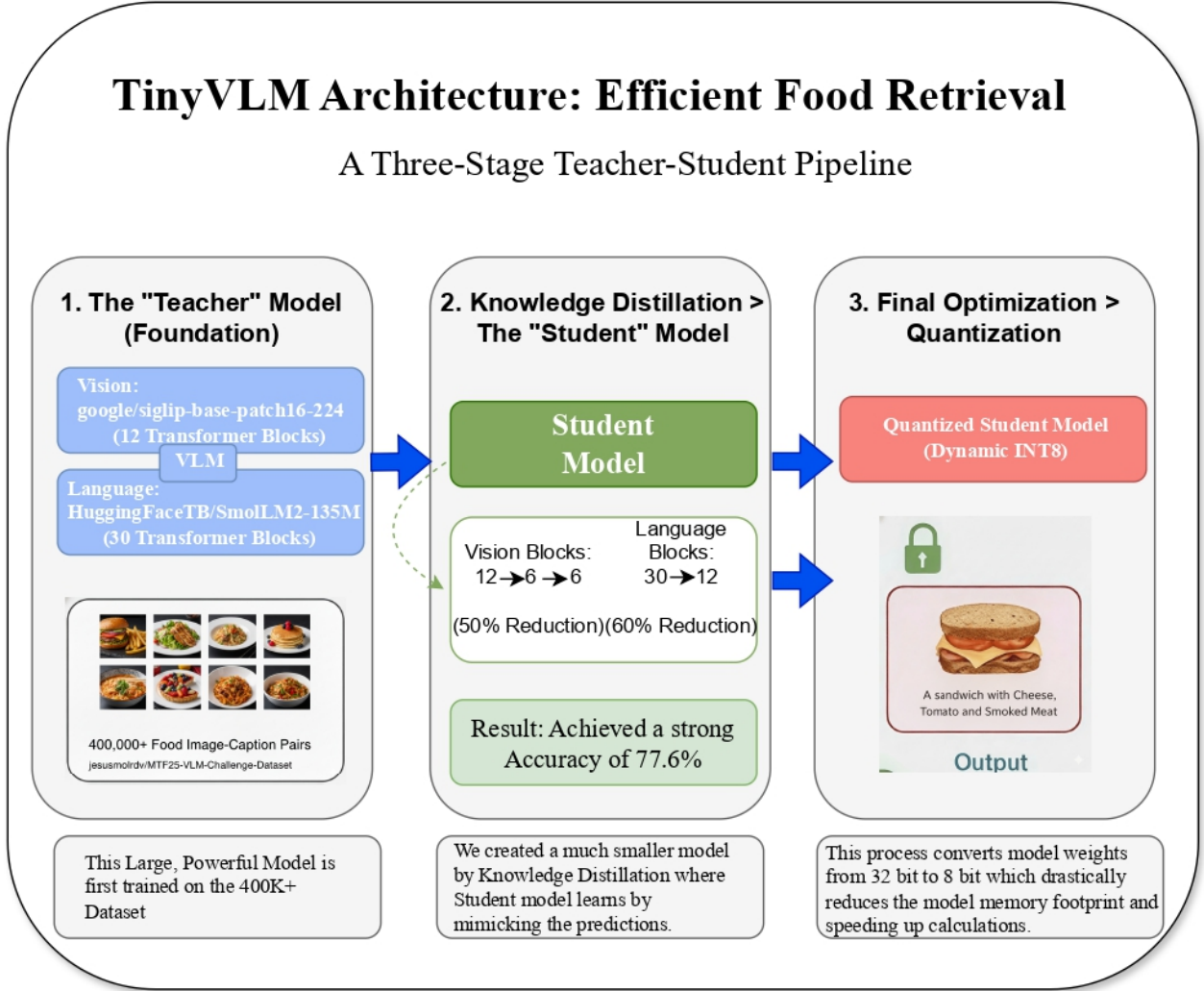


Fig. 1: The overall architecture of our proposed training and compression pipeline. (1) A large teacher VLM, built using SigLIP and SmolLM backbones, is trained on the food dataset using a contrastive loss. (2) A smaller student VLM is trained to mimic the teacher’s output similarity distribution via a distillation loss, in addition to its own contrastive loss. (3) The final distilled student model is quantized to 8-bit integers, producing the deployable TinyVLM.

knowledge distillation, and quantization [8]. Our work combines three of these: a compact network design for the student, knowledge distillation, and quantization, creating a synergistic pipeline for maximal efficiency.

C. Knowledge Distillation in VLMs

Knowledge distillation (KD) [9] is a powerful technique for transferring knowledge from a large teacher to a smaller student model. It has been successfully applied to compress VLMs by encouraging the student to mimic the teacher’s output logits or, more effectively, the relationships between modalities captured in the joint embedding space. This allows the student to learn richer patterns than it could from data labels alone.

D. Quantization of Neural Networks

Quantization reduces the numerical precision of a model’s parameters and activations, typically from 32-bit floating-point (FP32) to 8-bit integers (INT8) [10]. This leads to a substantial reduction in model size and can significantly accelerate inference on hardware with INT8 support. Post-Training Quantization (PTQ) is popular for its simplicity, as it does not require model retraining. We use dynamic PTQ, where weights are quantized offline and activations are quantized on-the-fly, offering an excellent balance of efficiency gains and ease of implementation.

TABLE I: Comparison of Teacher and Student Model Architectures

Parameter	Teacher Model	Student Model
Vision Encoder		
ViT Blocks	12	6
Hidden Dimension	768	288
Language Model		
LM Blocks	30	12
Hidden Dimension	576	288
Attention Heads	9	6
Total Parameters	220,312,128	50,145,216

III. METHODOLOGY

Our methodology is a multi-stage process designed to produce a highly efficient yet accurate VLM.

A. Model Architecture and Backbones

Both teacher and student models use a dual-encoder architecture. We leverage powerful, publicly available pre-trained models as backbones.

- **Image Encoder:** We use the SigLIP (google/siglip-base-patch16-224) model [6]. It is a Vision Transformer (ViT) pre-trained on a massive dataset with a sigmoid loss function, which has been shown to improve performance over the standard softmax-based contrastive loss.
- **Text Encoder:** We use SmoLLM-135M (HuggingFaceTB/SmoLLM2-135M) [7]. This is a compact, 135-million-parameter language model that provides a strong balance of performance and efficiency.

The student model is constructed with a shallower and narrower architecture compared to the teacher, as detailed in Table I, significantly reducing its parameter count.

B. Contrastive Training Objective

The core training objective is a symmetric InfoNCE loss. Given a batch of N image-caption pairs, we compute normalized image embeddings $I = \{i_1, \dots, i_N\}$ and text embeddings $T = \{t_1, \dots, t_N\}$. The loss is computed based on the cosine similarity matrix $S = I \cdot T^T$.

$$\mathcal{L}_{\text{contrastive}} = -\frac{1}{2N} \sum_{k=1}^N \left(\log \frac{e^{s_{k,k}/\tau}}{\sum_{j=1}^N e^{s_{k,j}/\tau}} + \log \frac{e^{s_{k,k}/\tau}}{\sum_{j=1}^N e^{s_{j,k}/\tau}} \right) \quad (1)$$

where τ is a learnable temperature parameter. This objective pushes matching image-text pairs together in the embedding space while pushing non-matching pairs apart.

C. Knowledge Distillation

When training the student model, we augment the contrastive loss with a distillation loss. Let S_T and S_S be the similarity matrices from the teacher and student models, respectively. The distillation loss minimizes the KL divergence between the student's and teacher's similarity distributions:

$$\mathcal{L}_{\text{distill}} = \text{KL}(\text{softmax}(S_S/T) \parallel \text{softmax}(S_T/T)) \quad (2)$$

where T is the distillation temperature. The final student training loss is:

$$\mathcal{L}_{\text{student}} = (1 - \alpha)\mathcal{L}_{\text{contrastive}}(S_S) + \alpha\mathcal{L}_{\text{distill}} \quad (3)$$

where α is a hyperparameter balancing the two objectives.

D. Post-Training Dynamic Quantization

After distillation, we apply dynamic quantization to the student model using PyTorch's native tools. This process converts the FP32 weights of the linear layers to INT8, significantly reducing the model's storage requirements. Activations are quantized dynamically during the forward pass, enabling the use of efficient integer-based matrix multiplication.

IV. EXPERIMENTS AND RESULTS

A. Dataset and Preprocessing

We used the Dishcovery: VLM MetaFood Challenge Dataset [11], combining its web and synthetic splits for a total of 364,535 training pairs and 19,185 validation pairs. Images are resized to 224x224 pixels and normalized. Captions are tokenized with a maximum length of 128.

B. Training Setup

All models were trained on a single NVIDIA RTX A2000 GPU with 12GB of VRAM, demonstrating that this pipeline is feasible without large-scale computational resources. We used the AdamW optimizer with a cosine learning rate scheduler and 500 warmup steps. Key hyperparameters include a batch size of 8, a distillation temperature $T = 2.0$, and a distillation weight $\alpha = 0.5$.

C. Quantitative Results

The models were evaluated on retrieval accuracy (averaged across image-to-text and text-to-image top-1 accuracy), model size, and simulated CPU inference latency to approximate an edge deployment scenario. The consolidated results are presented in Table II.

The findings provide clear answers to the research questions. In relation to RQ1, the compression pipeline substantially reduces the size of the original VLM while preserving a large portion of its retrieval capability. The distilled student model effectively recovers much of the performance lost due to architectural scaling, demonstrating that cross-modal similarity supervision is an effective mechanism for retaining fine-grained semantic alignment.

With respect to RQ2, the quantitative trade-offs achieved through the compression procedure are favorable for AIoT deployment. The final TinyVLM model attains a size of 111 MB, corresponding to an 86.9% reduction relative to the 847 MB teacher model. This compression is accompanied by a modest 7.8 percentage point decrease in accuracy. The simulated CPU latency improves from 94.7 ms for the teacher model to 84.9 ms for the quantized TinyVLM, representing a 10.3% reduction. These results indicate that the combination

of architectural scaling, distillation, and INT8 quantization yields a highly efficient model that maintains strong retrieval performance while aligning well with the resource and latency constraints of edge-oriented applications.

D. Qualitative Analysis: Capturing Finer-Grained Details

A key success of TinyVLM is its ability to understand not just broad food categories, but also the fine-grained details that distinguish different dishes. This is crucial for applications like nutritional analysis or recipe generation. As shown in Fig. 2, when presented with an image of a complex dish, TinyVLM retrieves captions that correctly identify specific ingredients and preparation methods, such as "grilled salmon," "creamy dill sauce," and "served with asparagus." This demonstrates that the knowledge distillation process successfully transferred the nuanced understanding of the larger teacher model, enabling the compact model to excel at fine-grained retrieval.

E. On-Device Deployment on Jetson Orin Nano

To assess the practical suitability of the compressed model for embedded AI scenarios, the distilled and quantized TinyVLM was deployed on an NVIDIA Jetson Orin Nano (8GB). The device integrates an ARM Cortex-A CPU cluster and an NVIDIA Ampere GPU with 1,024 CUDA cores within a unified memory architecture, making it representative of contemporary low-power AI edge platforms. All models were exported in PyTorch format and evaluated using a standardized benchmarking setup that measured inference latency, throughput, model storage size, and peak memory usage during execution. Inputs consisted of RGB images resized to 224×224 pixels and text captions truncated to 128 tokens. All evaluations were carried out in inference mode without gradient computation.

The inference performance on the Jetson Orin Nano is summarized in Table III. The quantized TinyVLM exhibits a latency of 1388.01 ms and achieves a throughput of 1.01 frames per second, representing a reduction of approximately 29.5% in latency relative to the full-precision teacher model and nearly a twofold improvement in throughput. This improvement highlights the computational advantages gained through the combined use of architectural scaling, distillation, and INT8 quantization.

Table IV reports the model sizes and peak memory usage on the device. The quantized TinyVLM occupies 111.11 MB of storage, compared to 847.17 MB for the full-precision teacher and student models, resulting in a $7.6\times$ compression ratio. This reduction substantially improves deployability on low-capacity embedded systems while retaining competitive performance. Peak memory usage increases for the quantized model due to runtime activation quantization, though the overall storage and latency characteristics remain strongly favorable for edge deployment.



www.bigstock.com · 44167111

(a) Query Image 1: Freshly prepared blt sandwich on wooden cutting board with knife. Tomatoes, lettuce, and mayo in background and toasted whole grain wheat bread in foreground. Macro with shallow dof.



(b) Query Image 2: BLT on Homemade Bread Stock Photography
Fig. 2: Qualitative results. TinyVLM retrieves captions that capture fine-grained details. For (a), retrieved captions could include "Grilled salmon with a creamy dill sauce and asparagus." For (b), captions could be "A juicy cheeseburger with melted cheddar and a glossy brioche bun."

Although the quantized model achieves the smallest storage footprint, its peak memory usage during inference is higher than that of the full-precision variants. This behavior is consistent with observations in prior work on post-training quantization [12], [13], where INT8 weights reduce persistent storage but activation tensors and intermediate computation buffers remain in floating-point precision. Dynamic quantization in PyTorch [14] stores parameters in INT8 format but performs dequantization of activations and matrix multiplications at runtime, producing temporary FP16 or FP32 buffers whose sizes dominate peak memory usage. In architectures with multi-branch modality encoders, such as dual-encoder VLMs, simultaneous processing of image embeddings, text embeddings, and similarity computations

TABLE II: Main Experimental Results Comparing Different Model Versions on the Dishcovery Validation Set

Model	Validation Accuracy (%)	Model Size (MB)	Relative Size	Simulated CPU Latency (ms)
Teacher VLM (FP32)	84.6	847.17	1.00x	94.7
Student VLM (from scratch, FP32)	71.2	847.17	1.00x	91.3
Student VLM (with Distillation, FP32)	77.6	847.17	1.00x	91.3
TinyVLM (Distilled + Quantized, INT8) (Ours)	76.8	111.11	0.13x	84.9

TABLE III: Inference Performance on Jetson Orin Nano

Model	Latency (ms)	Throughput (FPS)
Teacher VLM (FP32)	1967.72	0.51
Student VLM (FP32)	1875.33	0.52
TinyVLM (INT8, Quantized)	1388.01	1.01

TABLE IV: Model Size and Resource Utilization on Jetson Orin Nano

Model	Model Size (MB)	Peak Memory (MB)
Teacher VLM (FP32)	847.17	1293.54
Student VLM (FP32)	847.17	1244.53
TinyVLM (INT8, Quantized) (Ours)	111.11	2446.38

further increases transient allocation requirements [15]. As a result, peak runtime memory does not scale linearly with model size, even though permanent storage is substantially reduced.

Despite this peak memory overhead, the overall deployment profile of the quantized model remains significantly more favorable for embedded execution. The permanent storage requirement is reduced from 847 MB to 111 MB, enabling installation on devices with limited flash memory. Latency is reduced by 29.5%, throughput nearly doubles, and inference proceeds entirely within the thermal and power envelopes of the Jetson Orin Nano. These improvements demonstrate that the compressed TinyVLM maintains practical deployability even in scenarios where transient runtime memory peaks exceed the static model size, and they reinforce the suitability of the proposed compression strategy for real-time multimodal inference on edge platforms.

F. Quantization-Aware Training (QAT) Evaluation

An additional experiment was conducted using quantization-aware training (QAT) to determine whether further improvements could be achieved beyond post-training quantization. In this setting, the distilled student model was retrained with quantization operators inserted into the computation graph, allowing the network to adapt to the reduced numerical precision of INT8 arithmetic during optimization. The configuration included INT8 quantization for linear layers and floating-point quantization parameters for embedding modules to avoid instability. Due to the overhead associated with QAT and the need to preserve training feasibility on available hardware, the procedure was limited to a single fine-tuning epoch with a learning rate

TABLE V: Comparison of Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT)

Method	Val. Accuracy (%)	Model Size (MB)
PTQ (INT8)	76.8	111.11
QAT (INT8)	41.6	0.86

of 1×10^{-5} . This setup ensured that quantization observers remained active while minimizing disruption to the learned representations.

The results of this experiment are summarized in Table V. While the QAT-converted model achieved an extremely compact size of 0.86 MB, its retrieval accuracy reached 41.6 percent, substantially lower than the dynamically quantized model obtained through post-training quantization, which achieved 76.8 percent accuracy at a size of 111.11 MB. The discrepancy highlights the sensitivity of the compressed transformer-based student architecture to quantization noise during training. The reduced model capacity, combined with the multimodal nature of the network, makes it susceptible to destabilization when quantized constraints are introduced too early or too aggressively, particularly within attention blocks and projection layers that play a central role in aligning the vision and language embeddings.

These findings indicate that, within this specific architectural configuration, dynamic post-training quantization offers a more stable and deployment-ready solution. It maintains retrieval accuracy while avoiding the convergence challenges and additional training overhead associated with QAT. Although extended QAT schedules or layer-wise quantization strategies may ultimately narrow the gap, the present observations suggest that the combined approach of architectural reduction, contrastive distillation, and selective INT8 post-training quantization provides a more dependable balance of compactness, accuracy, and inference efficiency for edge-oriented vision-language applications.

V. CONCLUSION AND FUTURE WORK

This work presents a complete compression pipeline for transforming a large vision-language model into an efficient on-device system suitable for resource-constrained AIoT deployments. Through a sequence of architectural scaling, cross-modal knowledge distillation, and INT8 dynamic quantization, the resulting TinyVLM achieves substantial reductions in memory footprint and computational overhead

while maintaining strong retrieval accuracy. The model retains much of the representational capacity of its teacher despite operating at only 13% of the original storage size, illustrating the effectiveness of distillation-driven compression for fine-grained multimodal understanding.

Deployment on the Jetson Orin Nano further demonstrates the practical advantages of the compact model. The quantized TinyVLM exhibits markedly improved inference characteristics on embedded hardware, achieving lower latency and higher throughput compared to the full-precision teacher model. These findings confirm that the compressed architecture is well aligned with the operational constraints of real-time multimodal applications at the edge.

An exploration of Quantization-Aware Training (QAT) was also conducted to assess its potential for further accuracy recovery under low-precision constraints. Although QAT preserved numerical stability and achieved extremely compact INT8 representations, it did not produce accuracy improvements relative to the post-training quantized model in this setting. This outcome underscores that, for architectures of this scale and task specificity, dynamic post-training quantization already offers an effective balance between simplicity and performance.

Future work includes extending the deployment analysis to additional embedded platforms with stricter power and memory budgets and investigating hybrid quantization strategies that combine per-channel weight quantization with selective FP16 execution in high-sensitivity components. More advanced distillation objectives, such as alignment-aware or teacher-reweighted contrastive losses, may further enhance student performance and offer deeper insights into modality compression. Together, these directions aim to advance the development of robust, compact multimodal systems capable of supporting next-generation AIoT applications.

REFERENCES

- [1] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.
- [2] H. Cai, C. Gan, and S. Han, "Efficient deep learning for computer vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2021.
- [3] J. M. L-rdv, "nanovlm," 2024.
- [4] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning, ICML, 2021*.
- [5] C. Jia, Y. Yang, Y. Xia, Y.-T. Chen, Z. Parekh, H. Pham, Q. V. Le, Y. Sung, Z. Li, and T. Duerig, "Scaling up visual and vision-language representation learning with noisy text supervision," *arXiv preprint arXiv:2102.05918*, 2021.
- [6] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," 2023.
- [7] HuggingFaceTB, "Smollm2-135m," Hugging Face Model Card, 2024.
- [8] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "A survey of model compression and acceleration for deep neural networks," *arXiv preprint arXiv:1710.09282*, 2017.

- [9] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [10] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv preprint arXiv:1806.08342*, 2018.
- [11] M. VLMs, "Dishcovery: Vlm metafood challenge [cvpr'25]," 2025, kaggle.
- [12] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.
- [13] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv preprint arXiv:1806.08342*, 2018.
- [14] PyTorch Team, "Pytorch quantization documentation," 2023.
- [15] J. Zhang, H. Huang, Y. Li, C. Xu, Y. Tian, X. Hu *et al.*, "Vision-language models for vision tasks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.